

Randomized Algorithms for Primality Testing

A Journey from Exponential Traps to Probabilistic Certainty

Group ID: 231006

April 5, 2026

CS648: Randomized Algorithms

Act I: The Illusion of Efficiency

The Prime Suspect: Deterministic trial division up to $\sqrt{n}$.

At first glance, $O(\sqrt{n})$ sounds reasonable. But is it actually efficient?

The RAM Model Reality Check

In the RAM model of computation, an algorithm is only considered efficient if it runs in **polynomial time relative to the input size**.

- The input size to represent a number n in binary is its bit-length:
 $x \approx \log_2 n$.
- Therefore, the magnitude of the number is $n \approx 2^x$.

Act I: The Exponential Trap

The Grim Reality

Substituting our input size x into our time complexity gives us:

$$O(\sqrt{n}) = O\left((2^x)^{\frac{1}{2}}\right) = O\left(2^{x/2}\right)$$

The Verdict:

It's an exponential algorithm in disguise!

For a 1024-bit prime, it requires roughly 2^{512} operations. Even the world's fastest supercomputer would take longer than the age of the universe to finish.

We were staring at a computational dead end. We needed a clue...

Act II: The Preemptive Hint

The Intervention: Before we could even stress about the $O(2^{x/2})$ trap, the professor already knew exactly where we were headed. He stepped in with a preemptive directive.

“Don’t get stuck on trial division. Start looking at the intrinsic properties of primes. Dive deep into Fermat’s Little Theorem, plot the behavior, and see what the data tells you.”

Our Deep Dive: We stopped dividing and started exponentiating.

Fermat’s Little Theorem

If p is a prime number, then for any integer a such that $1 \leq a < p$:

$$a^{p-1} \equiv 1 \pmod{p}$$

The Hypothesis: If we pick a random base a , and $a^{n-1} \not\equiv 1 \pmod{n}$, then n is **definitely composite**.

Act II: Proving the Theorem (Part 1)

Trust, but Verify: To confidently use this theorem, we had to prove it.

Consider the set of non-zero integers modulo a prime p :

$$S = \{1, 2, 3, \dots, p-1\}$$

Multiply each element by our base a (where $1 \leq a < p$):

$$S' = \{a, 2a, 3a, \dots, (p-1)a\}$$

The Crucial Lemma

Because p is prime and $a < p$, no two elements in S' are congruent modulo p . Thus, S' is simply a **permutation** (a reshuffling) of S modulo p .

Act II: Proving the Theorem (Part 2)

Since S' is just a rearranged version of S modulo p , the product of all elements in each set must be congruent:

$$a \cdot 2a \cdot 3a \cdots (p-1)a \equiv 1 \cdot 2 \cdot 3 \cdots (p-1) \pmod{p}$$

Factor out the $(p-1)$ copies of a on the left side:

$$a^{p-1} \cdot (1 \cdot 2 \cdot 3 \cdots (p-1)) \equiv 1 \cdot 2 \cdot 3 \cdots (p-1) \pmod{p}$$

Recognize the factorials on both sides:

$$a^{p-1}(p-1)! \equiv (p-1)! \pmod{p}$$

Since p is prime, $(p-1)!$ shares no common factors with p . We can safely cancel it out to reach our conclusion:

The Result

$$a^{p-1} \equiv 1 \pmod{p}$$

Act III: The Empirical Challenge

The Next Meeting: We walked in confident. We had the math, and it was beautiful. But the professor wasn't ready to let us celebrate yet. He gave us a coding task.

*“Don't just trust the algebra. Write a script. Take all **composite** numbers n up to 1000. For each n , test every base $a < n$. Count how many bases satisfy the relation $a^{n-1} \equiv 1 \pmod{n}$.”*

The Goal: Hunting for Liars

We were looking for “Fermat Liars”—bases a that falsely testify that a composite number n is prime.

Ideally, this count should be zero, or at least very small.

We wrote the Python script, set n from 4 to 1000, and plotted the results...

Act III: The Plot Twist

The Results: For most composite numbers, the count of liars was comfortably low. But when we looked at the plot, we saw terrifying spikes.

The Carmichael Catastrophe

At $n = 561$, the graph exploded.

Even though $561 = 3 \times 11 \times 17$, it satisfied the equation $a^{560} \equiv 1 \pmod{561}$ for **every** base a coprime to it!

The Realization:

- These are **Carmichael numbers** (absolute pseudoprimes).
- If we blindly used Fermat's test, numbers like 561 would fool our algorithm almost every single time.
- Our beautiful randomized test was broken.

Act III: Visualizing the Betrayal

The Evidence: We plotted the number of bases a that falsely identified a composite n as prime.

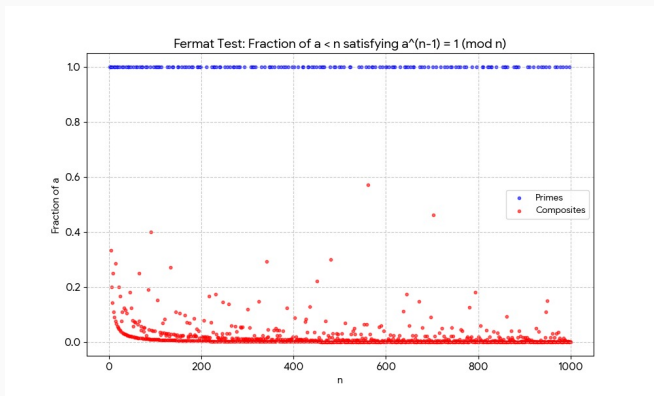


Figure 1: Frequency of Fermat Liars for Composite $n \leq 1000$

Act III: The Coprime Divide

The Next Meeting: We showed the professor our 561 anomaly. He smiled and told us to look deeper into the structure of the numbers we were testing.

“Don’t just look at a as a random integer. Look at its relationship with n . What happens when a shares a factor with n ? What happens when it doesn’t?”

Hypothesis 1: The Non-Coprimes ($\gcd(a, n) > 1$)

If a and n share a factor $d > 1$, then a will **always** reveal that n is composite.

The Proof: Assume for contradiction that $a^{n-1} \equiv 1 \pmod{n}$.

- This means $a^{n-1} - 1 = k \cdot n$ for some integer k .
- Rearranging: $a^{n-1} - k \cdot n = 1$.
- Since our common factor d divides both a and n , it must also divide the left side of the equation.
- Therefore, d must divide 1. But $d > 1$, which is a contradiction!

Act III: The Needle in the Haystack

The Good News: Any base a that is not coprime to n is a guaranteed "witness" to its compositeness. It will always fail the Fermat test.

The Bad News: The Coprime Issue

What if $\gcd(a, n) = 1$?

For a Carmichael number like 561, **every single coprime base** satisfies the relation $a^{560} \equiv 1 \pmod{561}$. They are all liars!

The Statistical Trap:

- To expose 561, our randomized algorithm **must** randomly select a base a that shares a factor with 561.
- But as n gets massive (e.g., the product of two 500-bit primes), almost every number below n is coprime to it.
- The non-coprime witnesses are so incredibly rare that picking one at random is like finding a needle in a haystack.

Act IV: The 50% Guarantee

The Next Meeting: We were discouraged by the massive haystack of coprimes. The professor pointed us to a hidden balance in the math.

*“If a number n is composite, and there is **at least one** coprime base that exposes it, look at what happens to the rest of the numbers. Assume you have one witness. What does that one witness do to the liars?”*

The Half-Liars Theorem

If there exists even a single coprime base w that fails the Fermat test ($w^{n-1} \not\equiv 1 \pmod{n}$), then **at least half** of all coprime bases will also fail the test!

Act IV: Proving the 50% Guarantee (Part 1)

Our Proof by Mapping: Let L be the set of "Liars" (coprimes where $x^{n-1} \equiv 1$) and let w be our one known "Witness" ($w^{n-1} \not\equiv 1$).

We created a simple function $f(x)$ that multiplies every liar $x \in L$ by our witness w :

$$f(x) = w \cdot x \pmod{n}$$

Step 1: The Output is a Witness

Test the output in Fermat's equation:

$$(wx)^{n-1} \equiv w^{n-1} \cdot x^{n-1} \equiv w^{n-1} \cdot 1 \equiv w^{n-1} \not\equiv 1 \pmod{n}$$

Result: Every liar mapped by w becomes a witness!

Act IV: Proving the 50% Guarantee (Part 2)

Step 2: The Mapping is One-to-One

Because w is coprime to n , $w \cdot x_1 \equiv w \cdot x_2$ implies $x_1 \equiv x_2$. No two liars map to the same witness.

The Conclusion

For every liar, there is a unique, distinct witness. Therefore, the number of witnesses must be $\geq$ the number of liars.

At most, only half the coprimes can be liars!

Act V: The Midsem Breather

Hitting the Wall: We had a 50% guarantee, but Carmichael numbers (with zero coprime witnesses) made it useless. We were stuck.

“You’ve hit the Carmichael wall. Take a breather. Appreciate just how tough this problem is. Over the midsem break, read up on Gary Miller and Michael Rabin. See how they bypassed this entirely.”

— The Professor

The Midsem Revelation: Over the break, we discovered the **Miller-Rabin Primality Test**. It doesn’t just rely on Fermat’s Little Theorem; it adds a second, devastatingly effective mathematical trap.

Act V: The Square Root Trap

The Miller-Rabin test relies on a fundamental property of algebra over prime fields.

The Non-Trivial Square Root Lemma

If p is a prime number, the equation $x^2 \equiv 1 \pmod{p}$ has exactly two solutions:

$$x \equiv 1 \pmod{p} \quad \text{and} \quad x \equiv -1 \pmod{p}$$

The Miller-Rabin Strategy:

- If we can find some number x where $x^2 \equiv 1 \pmod{n}$, but $x \not\equiv 1$ and $x \not\equiv -1$, then x is a **non-trivial square root of 1**.
- If a non-trivial square root of 1 exists modulo n , then n **must strictly be composite**.

Carmichael numbers can fool Fermat's equation, but they cannot hide from the square root trap!

Act V: The Mechanics (Step 0 & Step 1)

Step 0: The Trivial Case

Before doing any heavy math, we check if n is even. If $n > 2$ and is even, we instantly reject it as composite. The Miller-Rabin core logic is strictly for odd numbers greater than 3.

Step 1: Factoring out the Twos

Since n is an odd number, $n - 1$ must be an even number. We can factor out all the powers of 2 from $n - 1$ to write it as an odd number d multiplied by 2^s :

$$n - 1 = 2^s \cdot d \quad (\text{where } d \text{ is odd})$$

Why do we do this?

Fermat's Little Theorem looks at $a^{n-1} \equiv 1 \pmod{n}$. By rewriting the exponent, we are setting ourselves up to take the **square root** s times, working backwards from a^{n-1} down to a^d .

Act V: The Mechanics (Step 2 - The Sequence)

Step 2: The Repeated Squaring

We pick a random base a where $2 \leq a \leq n - 2$. We start by computing the base term $a^d \pmod{n}$. Then, we repeatedly square the result s times.

This generates a sequence of values:

$$a^d, \quad a^{2d}, \quad a^{4d}, \quad \dots, \quad a^{2^s d} \pmod{n}$$

Notice that the final term in this sequence is $a^{2^s d}$, which is exactly a^{n-1} .

- If n is prime, Fermat guarantees this final term is 1.
- But Miller-Rabin doesn't just care about the destination; it cares about the **journey** to get to 1.

Act V: The Verdict - Springing the Trap

How do we analyze the sequence to determine primality? We look for the non-trivial square roots of 1.

Condition for "Probably Prime"

The number n passes the test (acts like a prime) if either:

1. The sequence starts at 1: $a^d \equiv 1 \pmod{n}$.
2. The sequence hits -1 (which is $n - 1$) at some point before reaching 1.

Condition for "Definitely Composite"

If the sequence reaches 1, but the number **immediately preceding** the 1 in the sequence was NOT -1 , we have found a non-trivial square root of 1!

The trap is sprung. The number n is definitely composite, and even Carmichael numbers cannot escape this.

Act VI: The Ultimate Showdown

The Theory vs. The Reality: Math on a whiteboard is beautiful, but in CS648, the ultimate truth is in the computation. We needed to see if the trap actually worked.

The Final Bounty

We wrote a Python script to put our ultimate villains—the Carmichael numbers—back on trial.

- Would 561 survive the square root trap?
- What is the *actual* fraction of liars for the worst-case composites?

It was time to unleash Miller-Rabin and let the CPU do the talking...

Act VI: The 300,000-Number Grind

The Setup: We didn't just want to test a few edge cases. We wanted exhaustive, undeniable proof that the theoretical bound holds.

The Parameters

- **The Range:** We scanned every single odd composite number from 9 all the way up to 300,000.
- **The Test:** For *each* composite n , we tested **every single base** a from 2 to $n - 2$ to see if it passed or failed the Miller-Rabin test.

The Cost of Truth

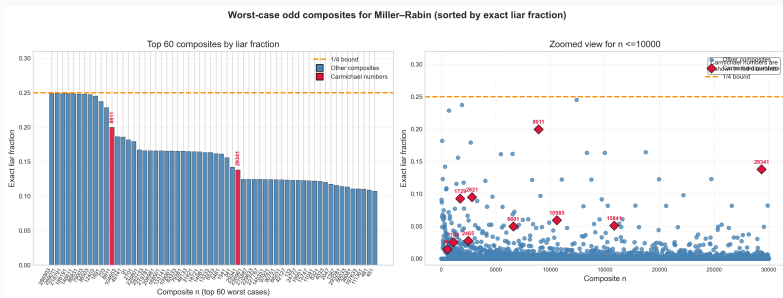
This exhaustive scan took about **5 to 6 hours** of continuous computation.

(If you heard what sounded like a jet engine taking off in the dorms recently, we apologize. That was just our laptop's cooling fan trying to survive the nested for-loops.)

Act VI: Visualizing the Worst Cases

Observation

This plot shows the exact Miller-Rabin liar fraction for the top 60 worst-case odd composites. Carmichael numbers are highlighted as red diamonds.



Act VI: Decoding the Results

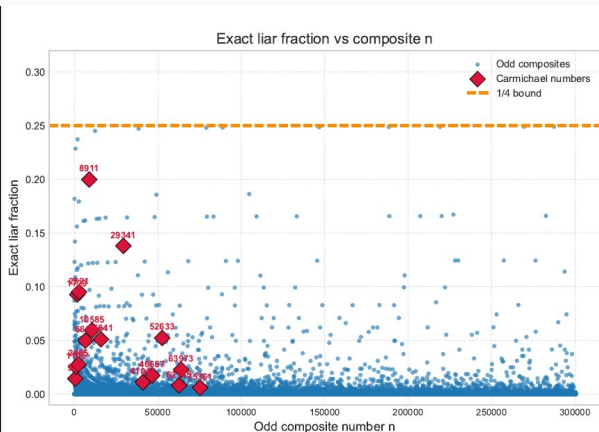
What the Data Tells Us: The plot reveals three massive insights about the power of the Miller-Rabin algorithm.

- **The 1/4 Bound is Absolute:** Look at the y-axis limit. Even for the absolute worst-case odd composites in our scan, the fraction of bases that act as liars never crosses 0.25. The mathematical guarantee holds perfectly.
- **The Fall of Carmichael Numbers:** The red diamonds represent the Carmichael numbers (like 561). Under Fermat's test, their liar fraction among coprimes was essentially 1.0. Here, they are entirely subdued well beneath the 1/4 line.
- **The Trap Works:** The data visually proves that while Carmichael numbers can bypass simple modular exponentiation, they cannot hide from the non-trivial square root check.

Act VI: A Closer Look at the Outliers

Interpretation

The second image gives a clearer zoom on the bad actors. Most composites cluster near zero liar fraction, while a few special numbers rise noticeably higher. Those points are exactly the ones we need to study, because they represent the hardest cases for the test.



Act VII: The Greed for Perfection

The Next Step: We had successfully bounded the error to $1/4$. But for computer scientists, a 25% worst-case failure rate is still terrifying. We wanted near-perfect certainty.

*“As sir always tells us: The key to randomized algorithms isn’t luck. It’s **repetition**.”*

The Amplification Experiment

We wrote a new simulation to test the power of repetition.

- We selected a random list of composite numbers (both Carmichael and non-Carmichael).
- For each number, we ran the Miller-Rabin test k times ($k = 1, 2, 3, 5, 10$).
- We ran massive thousands of trials to calculate the *exact* empirical probability of the algorithm failing.

Act VII: The Empirical Death of Error

Observation: Look at the most stubborn Carmichael numbers from our dataset. By $k = 10$, the error rate is completely annihilated.

Number (n)	Carmichael?	$k = 1$	$k = 2$	$k = 3$	$k = 5$	$k = 10$
8911	YES	0.1935	0.0455	0.0095	0.0005	0.0
29341	YES	0.1515	0.0220	0.0070	0.0000	0.0
2821	YES	0.1015	0.0090	0.0015	0.0000	0.0
1729	YES	0.0905	0.0045	0.0005	0.0000	0.0
10585	YES	0.0690	0.0035	0.0000	0.0000	0.0
6601	YES	0.0535	0.0040	0.0000	0.0000	0.0
52633	YES	0.0490	0.0015	0.0000	0.0000	0.0
15841	YES	0.0445	0.0040	0.0000	0.0000	0.0
4005	NO	0.0045	0.0000	0.0000	0.0000	0.0
4265	NO	0.0005	0.0000	0.0000	0.0000	0.0

Act VII: Escaping the Trap

The Scale of Certainty:

Our simulation proved the math: the probability of a composite surviving drops by at least 4^{-k} . If we run the loop just $k = 40$ times, the failure probability is $4^{-40} \approx 8.27 \times 10^{-25}$.

A hardware failure is statistically more likely than this algorithm failing!

The Ultimate Victory

Let's look back at where we started. Deterministic trial division trapped us in an impossible exponential time of $O(2^{x/2})$.

Miller-Rabin gives us near-absolute certainty in a blazingly fast polynomial time:

$$\text{Time Complexity} = O(k \log^3 n)$$

Conclusion: By willingly trading *absolute certainty* for *overwhelming probability*, we defeated the exponential trap.

Act VII: The Efficiency Reality Check

How fast is it really? We escaped the $O(2^{x/2})$ trap, but let's break down our final time complexity: $O(k \log^3 n)$.

Let $x = \log_2 n$ be the number of bits in our input.

- **One Multiplication:** Multiplying two x -bit numbers and taking the modulo takes $O(x^2)$ operations.
- **Repeated Squaring:** We perform at most x multiplications (squaring the base up to $n - 1$).
- **Single Trial:** $x \text{ steps} \times O(x^2) \text{ work} = O(x^3) =$

The Ultimate Contrast

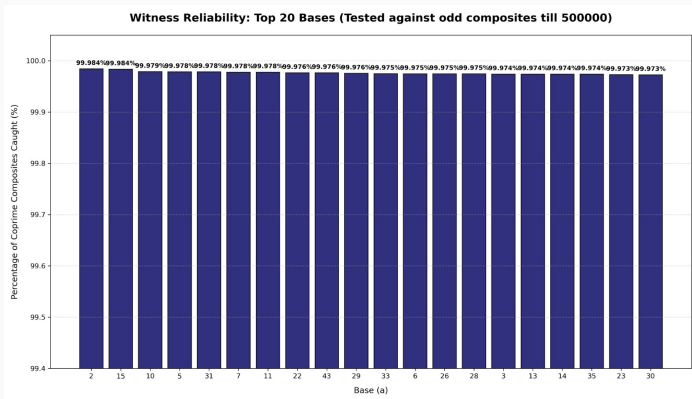
For a 1024-bit prime ($x = 1024$):

Trial Division: 2^{512} operations (*Longer than the age of the universe*).

Miller-Rabin ($k = 40$): $40 \times 1024^3 \approx 4.2 \times 10^{10}$ operations (*Fractions of a second on a laptop*).

Act VIII: Witness Reliability

Does the choice of base matter? Let's look at the data for the top 20 bases tested against odd composites up to 500,000.



Act IX: The Deterministic Bounds

Can we eliminate randomness? For most practical numbers, yes! If we test against specific sets of small prime bases, the Miller-Rabin test becomes 100% deterministic up to certain limits.

Bases Used	Correct for all $n <$
$\{2, 3\}$	1, 373, 653
$\{2, 3, 5\}$	25, 326, 001
$\{2, 3, 5, 7\}$	3, 215, 031, 751
$\{2, 3, 5, 7, 11, 13\}$	3, 474, 749, 660, 383
$\{2, 3, 5, 7, 11, \dots, 31, 37\}$	3.317×10^{24}

A Stunning Reality

We don't always need infinite trials. As the data shows, testing against just the first **12 prime bases** is enough to deterministically guarantee primality for all practical integers up to 3.317×10^{24} .

Beyond the Math: Our Learnings

To Our Professor:

You didn't just hand us the Miller-Rabin test on day one. You let us fail at trial division. You let us get betrayed by Fermat's illusion. You let us hit the wall of Carmichael numbers.

You didn't just teach us the algorithm; you made us respect the struggle it took to invent it.

The Philosophy of CS648

We walked into this project looking for a perfect equation, clinging to the need for absolute certainty. We leave with a profound realization:

In Computer Science, perfection is the enemy of progress. By willingly trading absolute certainty for overwhelming probability, we didn't compromise—we unlocked the universe of polynomial time.

Thank You

Questions?